A Calculus of Fidelity-Graded Translations

Manufacturing trust from unproved translators

(milestone snapshot, July 2026)

Anonymous Author(s)

Abstract

Verified translation has two well-studied extremes: prove the translator once (certified compilation), or validate each run of one translator (translation validation). Both treat a single translation in isolation. We study translations as a *graph* — many source languages, several reasoning targets, multiple independently built routes — where the honest answer to "is this translation correct?" differs from edge to edge. We present a calculus of *fidelity-graded translations*: pairs of languages close commuting squares that are checkable per program and compose by pasting; declared fidelity grades compose by weakest link, are *re-established* per run by inline checking, and are *exceeded* by agreement between independently derived routes; and an end-to-end theorem isolates a fundamental asymmetry — witness-carrying answers are self-certifying at the source, while universal answers are where grades, branches, and certificates earn their cost. The compositional core is mechanized in Lean 4.

The calculus is implemented in HURDY-GURDY, a platform of 13 languages and 13 pairs converging on two reasoning hubs, built as a *two-directional experiment in LLM-generated correctness*: independent LLM agents wrote every pair, largely unsupervised, with the architecture's cross-checks as the only semantic gate, and the platform's intended *player* is itself an LLM, handed deterministic, graded, checked moves. All code, and most of this paper, is LLM-generated; the human contribution is the architecture. We report per-construct coverage conjoined with fidelity, branch agreement along dual RISC-V and AArch64 routes, source-level witness replay, certified unreachability re-validated by a formally verified checker, and the defects the architecture caught — including one in that checker's own adapter.

1 Introduction

To answer a question about a program, move the program to where the question is decidable. A C program's reachability question becomes a bit-vector model-checking problem; a Python function's assertion becomes a linear-arithmetic query; a chemical reaction network's coverability question becomes an SMT unrolling. Every such move is a translation, and every translation is a place to be wrong. The classical responses are to *prove* the translator once and for all [7, 8], or to *validate* each of its runs [9, 13, 15]. Both are statements about a single edge.

This paper is about the *graph*. In practice the representations worth reasoning in form a web: several source languages, a few solver-facing targets, more than one way to get from here to there — and, crucially, edges of *honestly different* trustworthiness. A translator generated rule-for-rule from a logic-to-logic mapping is foreseeable from its specification. An optimizing C compiler is not foreseeable and not proved, only pinned. A translator written from an ISA manual and one derived from the same ISA's formal model are neither proved — but they were produced *independently*, and that independence is itself a resource. A theory of the graph must let each edge say what it actually guarantees, compose those statements honestly, and show how several weak edges can corroborate one another into a strong route. Its stance is the one distributed systems take toward processes: a translator is a component that fails, and the failure is to be *contained* — by declared loss, decidable checks, and independent routes — rather than prevented by universal proof.

An experiment in LLM-generated correctness, in both directions. This work is, deliberately, as much an experiment about *who can build and use* such a system as about the system itself. HURDY-GURDY — the platform this paper defines, builds, and measures — exists to show two things at once. First, that LLMs can be *utilized* to produce correct code: every pair — translator, carry-back, tests — was implemented by an independent LLM agent from a one-page brief, largely unsupervised, with the architecture's cross-checks as the only semantic gate; the platform is the experiment's artifact, and §6.5 reports what the gate caught. Second, that LLMs can be *supported* in producing correct conclusions: the platform's intended player is an LLM, which chooses questions and routes but can take no unchecked step — paths of pairs hand the untrusted reasoner deterministic moves whose trust is graded, cross-checked, and, for witness-carrying answers, self-certifying at the source. Building the instrument and playing it are the same thesis at two scales: trust comes from the architecture, not from the author.

Provenance disclaimer. All of HURDY-GURDY — framework, shared interpreters, pairs, solver plumbing, the evaluation harness, and the Lean mechanization — is LLM-generated code (Claude Opus and Claude Fable agents), produced largely unsupervised under the discipline this paper describes; no human reviewed the code for semantic correctness. Most of this paper's text was likewise LLM-generated. The human contribution is the architecture: the

calculus, the contracts the agents were held to, and the decisions about what to build and what to claim. Every quantitative claim in Section 6 is regenerated from the artifact by one script, and the mechanization's axiom audit is re-checked at every build — by design, the reader need not trust the writers, human or otherwise, any further than the paper's own thesis requires.

This paper. We give the graph that theory, and measure an implementation of it. The unit is the *pair* (Section 3): two formally defined languages, a pure translator, pure interpreters for both languages (owned by the languages and shared across all pairs touching them), and a pure *target-to-source interpreter* — the component, usually an afterthought, that carries a target-side behavior back to a source-side behavior. The four functions close a commuting square, and the square commuting *at a given program* is decidable: run both sides, compare under the pair's declared projection. A failure localizes to a step and a named observable.

Squares compose. The pasting of commuting squares is folklore; what is not folklore is the hypothesis it needs when each square commutes only *up to a projection*: the earlier hop's carry-back must not read target observables the later hop discards (Definition 3.6 and Theorem 3.7). This support condition is exactly the formal shadow of a design rule — *declare your loss* — and it yields the composed keep-set of a route: a source observable survives iff no hop discards what it depends on.

On this base we build the fidelity story (Section 4). Each edge declares a grade — `predicted`, `reproducible`, `checked`, `proved`, `trusted` — naming its guarantee and evidence. Grades compose by weakest link, but on an *assurance-class* chain (universal / per-run / replay / none) that separates the logic of a guarantee from the kind of evidence behind it. Two further mechanisms move assurance where weakest-link says it cannot go: *re-establishment* (Theorem 4.3) — inline square-checking lifts any single run over an opaque hop to per-run faithfulness, which is how a pinned-but-unproved compiler can head a verification route — and *branch corroboration* (Lemma 4.5) — independently derived routes to the same target that agree on the same input corroborate each other beyond either's declared grade, with disagreement localizing to one hop of one route (Lemma 4.4). A trusted-base ledger (§4.4) itemizes exactly what each layer of checking removes from the set of components one must still believe.

The capstone (§4.6) is an asymmetry we believe deserves to be a design principle. Answers that carry witnesses — REACHABLE, SAT — are *self-certifying at the source*: carry the witness back through the route's target-to-source interpreters and replay it in the source interpreter; soundness then rests on interpreter adequacy alone, with every translator and solver demoted to a discovery heuristic (Theorem 4.8). Universal answers — UNREACHABLE, UNSAT — admit no witness, and are precisely where fidelity grades, branch agreement, and independently checked certificates are the only currencies (Theorem 4.9). A translation architecture should therefore make witness carry-back a first-class component of every edge — cashing in the free half of trust — and spend its proof effort exclusively on the other half.

Coverage, and honesty about partiality. Fidelity is gameable by triviality (a one-instruction pair is vacuously `proved`); coverage is gameable by unsoundness. We require their per-construct *conjunction* against spec-enumerated inventories (Definition 4.6), with all rejection typed and enumerable, and we prove the widening discipline sound: additive extensions preserve all prior verdicts (Proposition 4.7), so coverage ratchets monotonically over a project's history.

The instantiation. The calculus is implemented as HURDY-GURDY, a platform of 13 registered languages and 13 pairs (Section 5): C, RISC-V, AArch64, Wasm, eBPF, EVM, Python, Sail, chemical reaction networks, SMILES molecular graphs, and two reasoning hubs — BTOR2 (bit-level) and SMT-LIB (theory-level) — plus dual, independently derived routes RISC-V→BTOR2 and AArch64→BTOR2 (manual-derived vs. Sail-model-derived), each pair agent-built as framed above with the square oracle, external differentials (a Sail-generated ISA simulator, a pinned CPython), solver corroboration, and branch cross-checks as the only police. Section 6 reports the resulting capability snapshot — per-pair conjoined coverage, composed route coverage, branch-agreement runs down to SMT-LIB, end-to-end case studies with source-level witness replay, certified unreachability, performance, and the defects the architecture caught along the way, each localized as Corollary 3.8 promises.

Contributions.

- A calculus of pairs: commuting squares with declared projections, decidable per-program checking with step/observable localization, and a pasting theorem whose projection-compatibility hypothesis makes cumulative loss compositional (Section 3).
- A fidelity algebra: grades vs. assurance classes, weakest-link composition, per-run re-establishment, branch corroboration under an explicit diversity assumption, and a trusted-base ledger (Section 4) — with the compositional core of both sections mechanized in Lean 4 (§4.7).
- The existential/universal asymmetry as an architectural principle, with the end-to-end soundness theorems that express it (§4.6).
- A measured, LLM-built instantiation: 13 agent-built pairs over 13 languages and two solver hubs, evaluated for conjoined coverage, branch agreement, end-to-end witness replay, certified unreachability re-validated

by a formally verified checker, and defects caught (Sections 5 and 6).

This document is a dated capability snapshot (July 2026) of a system whose coverage ratchets monotonically by construction; every number in Section 6 is regenerable from the artifact at this snapshot.

2 Overview: one question, two routes

Consider a small C program: a loop over an array with an index computation, and a question — can `x == 42` hold at the failure label? We narrate the route this question takes through the platform; Section 3 then makes the definitions precise.

The spine. The program first moves from C to RISC-V. The translator on this edge is a *pinned* C compiler: a digest-locked gcc with recorded flags. We can replay it bit-for-bit, but nobody can foresee or prove its output — its honest grade is `reproducible`, determinism without meaning. The RISC-V program then moves to BTOR2, a bit-vector transition-system logic that model checkers consume, and from there across a rule-for-rule bridge to SMT-LIB, where bit-vector solvers decide the reachability query.

Each of these moves is a *pair*: translator, source interpreter, target interpreter, and a target-to-source interpreter that carries behaviors backwards. When the RISC-V-to-BTOR2 translator runs, the platform can immediately check it *on this very program*: interpret the RISC-V program directly (the left edge of a square), interpret the BTOR2 translation and carry the resulting trace back to RISC-V observables (the other three edges), and compare, step by step, on the declared observables — program counter, registers, halt flag. That check passing is what the edge's `checked` grade *means*. It also retroactively covers the compiler: the compiled artifact is exactly what downstream squares validated on this run, so the opaque head is re-established per run (Theorem 4.3).

The branch. RISC-V reaches BTOR2 twice. One translator was written against the prose ISA specification; a second route factors through Sail, deriving its semantics from the official RISC-V Sail model. The two routes share no translator and no carry-back — only the endpoint languages. The platform runs the question down both, and the answers must agree. If they do, two independently derived encodings of RISC-V semantics corroborate each other — evidence strictly stronger than either route's own grade (Lemma 4.5). If they disagree, at least one route has a defect, and the per-hop squares localize it: which hop, which step, which observable (Lemma 4.4 and Corollary 3.8). The same dual-route structure exists for AArch64 (Arm manual vs. Arm Sail model).

The answer comes home. Suppose the SMT solver answers SAT: the label is reachable, with a model. The model is not the answer the user asked for — it is a valuation of SMT variables. The route's carry-backs now run in reverse: the SMT model becomes a BTOR2 witness, the witness a RISC-V trace, the trace an initial memory and register state — concretely, the inputs of the original C program. The platform then *replays* those inputs in the source interpreter and observes `x == 42` actually happen. At that moment the user's trust obligations collapse: even if every translator, both routes, and the solver were wrong, the replayed run exhibits the behavior (Theorem 4.8). Witness-carrying answers are self-certifying; only the interpreter that replays them — itself differentially tested against an independently generated ISA simulator — remains to be believed.

Had the solver answered UNSAT, no replay could vouch for it. That is where the grades, the branch agreement, the multi-engine corroboration, and the independently checked certificates carry the weight (Theorem 4.9) — and where the platform spends its assurance budget.

Beyond one spine. Nothing above is specific to C or RISC-V. The same square contract admits eBPF, Wasm, and EVM programs into the BTOR2 hub; Python functions and chemical reaction networks reach the SMT-LIB hub directly; a SMILES molecular graph reaches a molecular-formula target with no solver at all (a *compile pair* — the calculus does not care that the languages are far from computation, only that both have defined meaning). Each pair declares what it preserves (its projection), what it rejects (typed, enumerable `unsupported`), and its grade; the registry composes routes and reports each route's composed coverage, grade, and loss. The rest of the paper makes each of these words precise, then measures the system.

3 A calculus of pairs

This section makes the informal contract of Section 2 precise: languages, behaviors, and projections (§3.1); pairs and their commuting squares (§3.2); and the pasting theorem that lets squares compose into paths, together with the projection-compatibility condition that pasting actually requires (§3.3). Fidelity, coverage, and the end-to-end guarantee build on this base in Section 4.

3.1 Languages, behaviors, projections

Definition 3.1 (Language)**.** A *language* A is a triple $(\mathrm{Prog}_A, F_A, [\![\cdot]\!]_A)$ where Prog_A is a decidable set of *programs*; F_A is a finite set of named *observable fields*, each $f \in F_A$ with a value domain V_f; and $[\![\cdot]\!]_A : \mathrm{Prog}_A \rightharpoonup \mathrm{Beh}_A$ is the *reference semantics*, a partial function into behaviors. An *observable state* is a total map $\sigma \in \mathrm{Obs}_A$, where $\mathrm{Obs}_A = \prod_{f \in F_A} V_f$; a *behavior* is a finite sequence of observable states, $\mathrm{Beh}_A = \mathrm{Obs}_A^*$, recorded *post-step*: the i-th state is the observable state after the i-th transition.

The only admission criterion for a language is that $[\![\cdot]\!]_A$ is *definable* — an ISA manual, a logic's model theory, a reaction-network semantics, and a molecular-graph notation all qualify. Nothing in this section assumes languages are executable or even computational.

A program here is a *closed* configuration: free inputs, when present, are part of p (a program together with an input valuation). Open programs — programs quantified over their inputs — reappear in §4.6, where a solver decides over all inputs and a witness closes one.

Definition 3.2 (Interpreter)**.** An *interpreter* for A is a computable partial function $I_A : \mathrm{Prog}_A \rightharpoonup \mathrm{Beh}_A$ that is *pure*: identical input yields byte-identical output, on every host. Outside its domain it fails with a typed `unsupported` verdict naming the construct; $\mathrm{dom}(I_A)$ is A's *covered fragment*.

Assumption 1 (Interpreter adequacy)**.** $I_A(p) = [\![p]\!]_A$ for all $p \in \mathrm{dom}(I_A)$.

Adequacy is an assumption, not a theorem, and we keep it visible: it is the irreducible residue in every trusted-computing-base computation of §4.4. It is discharged *empirically*, by differential testing against an independently produced oracle for the same semantics (the Sail-generated RISC-V simulator; the pinned CPython runtime; Section 5), and its failure modes are cross-checked by the same machinery that checks translators.

Definition 3.3 (Projection)**.** A *projection* over A is a subset $\pi \subseteq F_A$. It lifts to states by restriction, $\pi(\sigma) = \sigma \restriction_\pi$, and to behaviors pointwise, $\pi(\sigma_1 \cdots \sigma_n) = \pi(\sigma_1) \cdots \pi(\sigma_n)$ (in particular, projected behaviors of different lengths are unequal). Write $b \equiv_\pi b'$ for $\pi(b) = \pi(b')$.

3.2 Pairs and the commuting square

Definition 3.4 (Pair)**.** A *pair* $P : A \to B$ is a tuple $(A, B, T, \Lambda, \pi_P)$ where $T : \mathrm{Prog}_A \rightharpoonup \mathrm{Prog}_B$ is the *translator*; $\Lambda : \mathrm{Beh}_B \rightharpoonup \mathrm{Beh}_A$ is the *target-to-source interpreter*, which re-expresses a target behavior as a source behavior; and $\pi_P \subseteq F_A$ is the pair's *declared projection* — exactly the source observables it promises to preserve. T and Λ are pure (as in Definition 3.2); I_A and I_B are owned by the languages and shared by every pair that touches them. The pair's domain $\mathrm{dom}(P)$ is the set of p with $p \in \mathrm{dom}(I_A) \cap \mathrm{dom}(T)$, $T(p) \in \mathrm{dom}(I_B)$, and $I_B(T(p)) \in \mathrm{dom}(\Lambda)$.

Definition 3.5 (Faithfulness)**.** P is *faithful at* $p \in \mathrm{dom}(P)$ when the square commutes at p, i.e. when $\pi_P(I_A(p))$ equals $\pi_P(\Lambda(I_B(T(p))))$:

$$\begin{array}{ccc} p & \xrightarrow{\quad T \quad} & T(p) \\ {\scriptstyle I_A}\downarrow & & \downarrow{\scriptstyle I_B} \\ I_A(p) & \xleftarrow{\equiv_{\pi_P}} & \Lambda(I_B(T(p))) \end{array}$$

P is faithful on $C \subseteq \mathrm{dom}(P)$ when it is faithful at every $p \in C$.

Three remarks. First, everything in Definition 3.5 is computable: faithfulness at a given p is a *decidable, executable check* — run both sides, compare under π_P. We call this check the *square oracle*; in distributed-systems terms it is a failure detector, turning a silently wrong translator into a *fail-stop* component [18]. Second, when the check fails it fails at a *first* position: a least step index i and a field $f \in \pi_P$ on which the two sides differ. The oracle reports (i, f) — a translator (or interpreter, or carry-back) defect, localized to a step and a named observable. Third, the projection is part of the contract, not a tuning knob: π_P states what "preserves meaning" promises for this pair, and $F_A \setminus \pi_P$ — the pair's *loss* — states what it does not.

The carry-back Λ is what makes a target-side result *mean* something at the source: a solver counterexample or a target trace is re-expressed as a source behavior. It is pair-specific because the correspondence it encodes is pair-specific; it is also why the square's oracle is as expressive as the translator it checks.

3.3 Pasting: when squares compose

Two pairs compose when the middle language is shared: $P_1 = (A, B, T_1, \Lambda_1, \pi_1)$ and $P_2 = (B, C, T_2, \Lambda_2, \pi_2)$ yield the candidate composite

$$P_2 \circ P_1 \;=\; (A,\; C,\; T_2 \circ T_1,\; \Lambda_1 \circ \Lambda_2,\; \pi)$$

for a projection $\pi \subseteq \pi_1$ to be determined, with $\mathrm{dom}(P_2 \circ P_1)$ the evident composite domain. Folklore says the squares paste. They do not paste for free: P_2 guarantees agreement of the two B-behaviors only *up to* π_2, and Λ_1 consumes whole B-behaviors. If Λ_1 reads a B-field that P_2 discards, the outer rectangle can fail while both inner squares hold. The missing condition is that Λ_1 must not look outside what P_2 preserves:

Definition 3.6 (Support)**.** Λ_1 is $(\pi_2 \Rightarrow \pi)$*-supported* when for all $b, b' \in \mathrm{dom}(\Lambda_1)$: $\pi_2(b) = \pi_2(b')$ implies $\pi(\Lambda_1(b)) = \pi(\Lambda_1(b'))$, and moreover $\mathrm{dom}(\Lambda_1)$ is π_2-closed on the behaviors compared ($b \in \mathrm{dom}(\Lambda_1)$ and $\pi_2(b) = \pi_2(b')$ imply $b' \in \mathrm{dom}(\Lambda_1)$).

Theorem 3.7 (Pasting)**.** *Let $p \in \mathrm{dom}(P_2 \circ P_1)$, let P_1 be faithful at p, let P_2 be faithful at $T_1(p)$, let $\pi \subseteq \pi_1$, and let Λ_1 be $(\pi_2 \Rightarrow \pi)$-supported. Then, writing $r = T_2(T_1(p))$, the composite $P_2 \circ P_1$ is faithful at p with respect to π:*

$$\pi(I_A(p)) \;=\; \pi\big(\Lambda_1(\Lambda_2(I_C(r)))\big).$$

Proof. Write $q = T_1(p)$. Then $\pi(I_A(p)) = \pi(\Lambda_1(I_B(q)))$ since $\pi \subseteq \pi_1$ and P_1 is faithful at p. Faithfulness of P_2 at q gives $\pi_2(I_B(q)) = \pi_2(\Lambda_2(I_C(r)))$, so the support condition applies to the two B-behaviors and yields $\pi(\Lambda_1(I_B(q))) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$. Chain the equalities. (Details: Appendix A.) □

Composed keep and loss. In the implementable special case — Λ_1 *fieldwise* and step-aligned, each source field f computed from a dependency set $\mathit{deps}(f) \subseteq F_B$ of target

fields — the support condition has a syntactic reading, and the largest π satisfying Theorem 3.7 is

$$\mathsf{keep}(P_2 \circ P_1) \;=\; \{\, f \in \pi_1 \mid \mathit{deps}(f) \subseteq \pi_2 \,\},$$

a source field survives the path iff P_1 keeps it and it is computed only from target fields P_2 keeps. Dually $\mathsf{lost}(P_2 \circ P_1) = F_A \setminus \mathsf{keep}(P_2 \circ P_1)$: path loss is the union of per-hop losses, each pulled back along the carry-backs. This is the formal content of the platform rule that a path must *declare its cumulative loss*: the observables a destination can still speak about are exactly those no hop discarded. (When Λ_1 changes step granularity — one source step spanning several target steps, as in C-to-ISA — the same statement holds with *deps* taken over the spanned window; Appendix A.)

Corollary 3.8 (Localization)**.** *Under the support condition, if the composite square fails at p (both sides defined), then P_1's square fails at p or P_2's square fails at $T_1(p)$. Inductively, along a route $R = P_n \circ \cdots \circ P_1$ satisfying the pairwise support conditions, a failing outer rectangle implies a failing inner square at some hop i — and running the square oracle hop by hop finds the least such i, then the least failing step and field.*

Corollary 3.8 is the debugging story of the whole platform: a divergence anywhere along a route is never a mystery about the route; it is an indictment of one hop, one step, one observable. It is also what disagreement between *routes* will lean on in §4.3.

Routes. The registry of languages and pairs forms a directed multigraph; a *route* $R : A \longrightarrow Z$ is a path through it, with composed translator T_R, composed carry-back Λ_R, composed projection $\mathsf{keep}(R)$, and faithfulness given by iterating Theorem 3.7. Purity composes too, and gives caching:

Proposition 3.9 (Determinism and caching)**.** *If every component function of every hop of R is pure, then T_R and Λ_R are pure, and a content-addressed cache keyed by the input hash and the component versions returns, on a hit, exactly what recomputation would return — across the whole route. If some hop is impure, re-running the route on the same input and diffing bytes at every hop (*twice-and-diff*) detects it and localizes it to the first hop whose bytes differ.*

4 Fidelity, coverage, and the end-to-end guarantee

Section 3 says what it means for one translation, or one route, to be faithful. This section grades *how well we know* it is faithful (§4.1), shows how the grades compose — weakest link, per-run re-establishment, and branch corroboration (§4.2–§4.3) — accounts for what each layer of cross-checking removes from the trusted base (§4.4), pairs fidelity with its anti-triviality companion, coverage (§4.5), and assembles everything into the end-to-end guarantee a user actually receives (§4.6).

4.1 Fidelity grades and assurance classes

A pair declares a *fidelity grade* from the set $G = \{$`trusted`, `reproducible`, `checked`, `predicted`, `proved` $\}$, each grade naming both a guarantee and the *kind of evidence* that establishes it: `predicted` — the translator's output is derivable byte-for-byte from a written specification (evidence: an audit that re-derives the bytes); `proved` — a machine-checked certificate that the square commutes, re-checked by an independent checker (evidence: the certificate and the checker's pedigree); `checked` — the square oracle runs and passes on every translation actually performed (evidence: this run's oracle verdict); `reproducible` — determinism only, via a digest-pinned toolchain (evidence: replayability, no meaning guarantee); `trusted` — nothing.

The grades order by evidence strength, but `predicted` and `proved` are incomparable in *kind* — one is foreseeable, the other re-checkable — and nothing below is gained by forcing them into a line. What composition actually needs is the logical *form* of the guarantee, and that is totally ordered:

Definition 4.1 (Assurance class)**.** The *assurance classes* are the chain $\mathbb{A}$: none $<$ replay $<$ perrun $<$ universal, and $\mathit{class} : G \longrightarrow \mathbb{A}$ maps `trusted` $\mapsto$ none, `reproducible` $\mapsto$ replay, `checked` $\mapsto$ perrun, and `predicted`, `proved` $\mapsto$ universal. The classes denote guarantee predicates for a pair P with covered fragment C_P: universal — P is faithful at every $p \in C_P$; perrun — P is faithful at every p on which the square oracle has run and passed (in particular, at the program of the present run); replay — T is pure, nothing about meaning; none — nothing.

Separating G from $\mathbb{A}$ resolves a wrinkle the informal five-tier story leaves open (what is the "meet" of `predicted` and `proved`?): grades differ in evidence, classes in logic, and only classes compose. A route with one `predicted` hop and one `proved` hop is universal end to end, as it should be.

4.2 Composition: weakest link, and per-run re-establishment

Proposition 4.2 (Weakest link)**.** *Let route $R = P_n \circ \cdots \circ P_1$ satisfy the support conditions of Theorem 3.7. If every hop's guarantee holds, then the composite guarantee of class $\min_i \mathit{class}(P_i)$ holds for R (with C_R the composite fragment), and no higher class is entailed by the hop guarantees alone.*

The proof is immediate from Definition 4.1 and Theorem 3.7: universal statements conjoin into universal statements over the composite fragment; a per-run statement at hop i caps the conjunction at the programs the oracle actually saw; a replay hop caps it at purity. Optimality is by the evident counterexamples. Statically, then, *a route is as faithful as its weakest hop* — but the static story is not the whole story, because every pair carries its own oracle:

Theorem 4.3 (Per-run re-establishment)**.** *Fix a run of route R on program p, with intermediate programs $q_0 = p$, $q_i =$*

$T_i(q_{i-1})$. *If on this run the square oracle runs and passes at every hop i (on q_{i-1}), and the support conditions hold, then R is faithful at p — regardless of the hops' static grades. In particular a* `reproducible` *or even* `trusted` *hop that is dynamically validated at q_{i-1} contributes, for this run, exactly what a* `checked` *hop contributes.*

Proof. Each passed oracle check *is* faithfulness at q_{i-1} (Definition 3.5 is decidable and the oracle decides it); apply Theorem 3.7 inductively. □

Theorem 4.3 is why an opaque, pinned C compiler is admissible at the head of a verification route: its static grade is `reproducible` (honest: nobody can foresee or prove its output), yet every run's result is validated downstream, lifting *that run* to perrun. The declared route grade remains the weakest-link meet — re-establishment is per-run evidence, never a static promotion.

4.3 Branch corroboration

The registry graph is not a line: one source may reach one destination by several routes. Let $R_1, R_2 : A \longrightarrow Z$ with composed keep-sets K_1, K_2 and $K = K_1 \cap K_2$, sharing the endpoint interpreters (language-owned) but no translator or carry-back.

Lemma 4.4 (Disagreement localizes)**.** *If R_1 and R_2 disagree at p under K, at least one route is unfaithful at p, and Corollary 3.8 pins the failure to a hop, a step, and a field of that route.*

Lemma 4.5 (Agreement corroborates)**.** *If R_1 and R_2 agree at p under K, then either both are faithful at p (under K), or both are unfaithful with* identical projected error*: defects in independently built components producing the same wrong K-behavior at p.*

Both are immediate: two faithful routes each equal $K(I_A(p))$, hence each other. What Lemma 4.5 buys depends on an assumption we state rather than hide:

Assumption 2 (Diversity)**.** Corroborating routes share no translator or carry-back, and their translators are derived from *independent semantic artifacts* — e.g. RISC-V reaches BTOR2 once via a translator written from the prose ISA specification and once via the Sail formal model; likewise AArch64 from the Arm manual vs. the Arm Sail model.

Under Assumption 2, the residual failure mode of an agreeing branch is a *common-mode failure*: independently produced encodings of the same semantics, wrong in the same observable way on the same program. We do not assign this a probability; we account for it structurally, in the trusted base ledger below. This is how the platform manufactures high assurance out of hops that are individually only `checked` — dual modular redundancy whose comparator, unlike a voter, localizes: agreement of independent routes is a stronger fact than either route's grade.

Table 1. The trusted-base ledger: what each layer of cross-checking removes from the set of components a verdict still assumes correct.

Layer that ran	**Removed from** TCB	**Still in** TCB
bare route + solver	—	all T_i, Λ_i, all interpreters, solver
+ square oracle at every hop (Theorem 4.3)	every T_i	the Λ_i, the interpreters, solver
+ branch agreement (Lemma 4.5, Assumption 2)	per-route Λ_i (cross-corroborated)	shared endpoint interpreters, solver; common-mode residue
+ independent solver corroboration	the single solver	the agreeing engines' common failure modes
+ certificate check (`proved`; e.g. DRAT/LRAT)	the solver entirely	the certificate checker (formally verified, in the `cake_lpr` instantiation), plus the bit-blaster's BV→CNF step
+ source-level witness replay (§4.6)	*everything above*	I_A adequacy (Assumption 1) alone

4.4 The trusted-base ledger

Define the *trusted computing base* TCB(v) of a verdict v as the set of components whose correctness v's soundness still assumes after all checks that ran. Each cross-check layer strictly shrinks it (Table 1); the last two rows are not interchangeable, and the difference is the central asymmetry of the whole design.

4.5 Coverage, and the ratchet

Fidelity alone is gameable by triviality: a pair handling one instruction is vacuously `proved`. Its companion axis is measured against a yardstick the implementer does not choose:

Definition 4.6 (Coverage)**.** Fix for each language a *construct inventory* K_A enumerated from its specification (opcodes, operators, syntactic forms). A construct $k \in K_A$ *counts* for pair P iff P covers it (k's probe programs lie in $\mathrm{dom}(P)$) *and* P is faithful on those probes: $\mathrm{cov}(P) \subseteq K_A$ is the per-construct *conjunction* of covered and faithful. Route coverage $\mathrm{cov}(R)$ counts k iff its lowering survives — covered and faithful — at every hop.

Coverage is gamed by unsoundness (accept everything, translate it wrongly) exactly as fidelity is gamed by triviality (translate one thing, prove it); the conjunction closes both routes to a vacuous "pass." Everything a pair rejects must be a *typed* `unsupported`, so the uncovered set is itself enumerable and auditable — partiality is fail-fast, never silent.

Coverage then grows by a discipline with a soundness theorem:

Proposition 4.7 (Ratchet)**.** *Call an update $I' \sqsupseteq I$ (likewise T, Λ) an* extension *when* $\mathrm{dom}(I') \supseteq \mathrm{dom}(I)$ *and* $I' \restriction_{\mathrm{dom}(I)} = I$*. Under extensions: every faithfulness verdict at every $p \in \mathrm{dom}(I)$ is preserved;* cov *is monotone; and prior evidence (oracle passes, differentials, agreement records) never needs re-earning. An update that is* not *an extension can silently invalidate any of these, and must instead bump the component's version and re-validate every dependent pair.*

The implementable extension check is byte-identity of all old outputs across the update — the same twice-and-diff mechanism as Proposition 3.9, run across versions. This is precisely the repo's widening protocol (additive interpreter bumps, versioned events, dependent-pair re-validation), which Section 6 measures over the project's own history.

4.6 The end-to-end guarantee

Finally, what does a user hold at the end? Let $R : A \longrightarrow Z$ be a route into a *reasoning language* Z (one a solver consumes: a transition-system logic, an SMT theory). The user asks a question about an *open* source program $p(x)$ — free inputs x — and a condition φ over $\mathrm{keep}(R)$-observables: is φ reachable? The solver decides at Z; its answer comes back in one of two logical shapes, and they differ fundamentally in what must be trusted.

Theorem 4.8 (Existential answers are self-certifying)**.** *Suppose the solver returns* REACHABLE *with witness w, the composed carry-back Λ_R re-expresses w as an input valuation x_0 (and claimed trace), and replaying $I_A(p(x_0))$ exhibits φ. Then φ is truly reachable in $[\![p]\!]_A$ — with* TCB = {*Assumption 1 for I_A*} *and nothing else. Every translator, carry-back, hub interpreter, and solver on R is a* discovery *device only: a defect in any of them can cause a missed or non-replaying witness, never a false* REACHABLE*.*

Proof. The replay is a run of I_A itself on a concrete closed program $p(x_0)$; by Assumption 1 its behavior is $[\![p(x_0)]\!]_A$, and φ was observed on it. No other component's output enters the final judgment. □

Theorem 4.9 (Universal answers need the machinery)**.** *Suppose the solver returns* UNREACHABLE *(within declared unrolling/step bounds k). If (i) every hop of R carries a* universal*-class guarantee on the fragment containing p's constructs, or carries* perrun *evidence together with branch agreement per Lemma 4.5; (ii) the support conditions of Theorem 3.7 hold and φ mentions only* keep(R)*; and (iii) the verdict is corroborated across independent engines or certified by an independently checked certificate, then no input drives $[\![p]\!]_A$ to φ within k steps, with* TCB *as in the corresponding row of the ledger (§4.4).*

The asymmetry between Theorems 4.8 and 4.9 is, we argue, the right lens on the entire design: *for questions whose answers carry witnesses, trust is nearly free* — carry the witness back and replay it at the source; the platform's graded tiers, branches, and certificates exist for the other half, the universal answers where no witness can vouch for the verdict. This is the end-to-end argument [17] transplanted to translation: the check that settles a witness-carrying answer belongs at the endpoint — the source interpreter — and what the intermediate hops provide is, for that half, an optimization; only for universal answers, where no endpoint check can exist, does hop fidelity become the guarantee itself. A system that makes the carry-back Λ a first-class, per-pair component is a system in which the cheap half of trust is always cashed in; a system that grades and stacks cross-checks is one in which the expensive half is bought consciously, in labeled increments, with the bill itemized in the ledger of §4.4.

4.7 Mechanization

The compositional core of Sections 3 and 4 is mechanized in Lean 4 (core library only, no mathlib; ~750 lines, in the artifact): the pasting theorem with the support condition, localization, the assurance-class chain and weakest link (sound half), per-run re-establishment, both branch lemmas, the ratchet (verdict preservation and coverage monotonicity under extensions), both end-to-end theorems, and the *route telescope* — routes of arbitrary length as a dependently-typed chain of pairs over interpreter-bundled languages, with the paper's "pairwise support conditions" as a recursive coherence predicate, and the n-ary pasting and localization theorems proved by induction through the binary ones (the canonical head-projection choice at each junction is shown lossless by a reprojection lemma). Three modeling choices mirror the paper exactly. Components are Lean functions, so *purity is by construction* and Proposition 3.9 is definitional; partiality is `Option`, with `unsupported` as `none`; and projections are generalized from field subsets to arbitrary observable-compression maps, with "$\pi \subseteq \pi_1$" becoming *factoring*. Because every theorem takes each component-output witness as an explicit hypothesis, a statement's trusted base is literally its hypothesis list — Theorem 4.8 in Lean assumes source-interpreter adequacy and nothing else, and its proof term is three tokens. The axiom audit is re-checked at every build: the branch lemmas, the ratchet, and Theorem 4.8 are axiom-free; pasting, weakest link, and Theorem 4.9 use only `propext`; the one classical proof is Corollary 3.8 (an unfaithful route names no witness by itself). Deliberately not mechanized: the fieldwise loss computation and retiming case of Appendix A (syntactic side conditions over a concrete carry-back representation), the optimality halves of Proposition 4.2 (meta-statements over models), and the grade set G itself — evidence provenance is not a mathematical object, which is the point of separating G from $\mathbb{A}$. The telescope theorems' audit adds only `Quot.sound` (from structural-recursion equations); localization remains the sole classical proof.

5 Instantiation: the platform

The calculus is implemented as HURDY-GURDY, a Python platform whose structure mirrors Section 3 exactly: a *framework* that holds no pair semantics, per-language *shared interpreters*, and per-pair translators and carry-backs. This section describes what exists at the snapshot; Section 6 measures it.

5.1 Framework

The framework provides the registry (the language/pair graph with deliverable status); the content-addressed cache of Proposition 3.9; the generic square oracle — align $I_s(p)$ against $\Lambda(I_t(T(p)))$ under π, with step/observable localization —; the route enumerator and path runner (routes are enumerated and reported with composed grade, coverage, and loss — never chosen; choosing is the caller's job); the coverage harness (construct-inventory extraction, typed-`unsupported` histograms); the path grader (composed determinism, composed coverage, and branch-agreement measurements, run on merge); and the solver/checker plumbing. Everything except the solver call is byte-deterministic, and a twice-and-diff harness enforces Proposition 3.9 repo-wide.

5.2 Languages and reasoning hubs

Thirteen languages are registered, each with a specification-anchored formal semantics and (for eleven of them, at the snapshot) a built shared interpreter: the ISAs RISC-V (RV64IMC), AArch64 (a growing A64 slice), eBPF, Wasm, and EVM; C (whose "interpreter" role is played differentially, see below); Python (a pinned CPython restricted by an AST allow-list to an integer subset — the loader rejects, CPython runs); Sail (an executable slice of the Sail-derived RISC-V and Arm semantics); chemical reaction networks (a discrete Petri-net stepper); SMILES molecular graphs and molecular formulas; and the two *reasoning hubs*: BTOR2, a bit-vector/array transition-system logic consumed by hardware model checkers, and SMT-LIB (QF_ABV and QF_LIA fragments).

Reasoning languages own, beyond an interpreter, a solver inventory (BtorMC and Pono for BTOR2; Z3, Bitwuzla, Boolector, cvc5, Yices2 for SMT-LIB) and a witness-checker inventory: interpreter replay of BTOR2 `.wit` witnesses, model evaluation by the deterministic SMT-LIB evaluator, multi-engine corroboration, and a bit-blasted certificate pipeline for `proved`-tier unreachability with two checker rungs: the DRAT proof checked by `drat-trim` (independent, unverified), and — preferred when present — elaborated to LRAT (`drat-trim` as an *untrusted* elaborator) and re-validated by `cake_lpr`, the *formally verified* CakeML checker, built natively from its compiler-generated ARMv8 assembly (§6.4).

Interpreter adequacy (Assumption 1) is discharged differentially: the RISC-V interpreter against the Sail-generated reference simulator (`sail_riscv_sim`) and the official `riscv-tests` / `riscv-arch-test` suites; the Python subset against pinned CPython by construction; C against CBMC on the same source (with a classification step separating C-undefined-but-ISA-defined behavior from genuine faults).

5.3 Pairs and routes

Thirteen pairs are built to varying, *measured* extents (Table 2). The spine is C → RISC-V → BTOR2 → SMT-LIB, with the C head a digest-pinned gcc at grade `reproducible`, re-established per run per Theorem 4.3. RISC-V and AArch64 each reach BTOR2 by two independently derived routes (manual-derived direct translator vs. Sail-model-mediated), instantiating Assumption 2; the BTOR2 hub funnels five ISAs into one solver interface, and the BTOR2 → SMT-LIB bridge (a rule-for-rule operator mapping at grade `predicted`) connects the hubs, so every BTOR2 question can also be decided natively-vs-bridged — solve-step corroboration. CRN and Python reach SMT-LIB directly (QF_LIA), and SMILES→formula exercises the calculus away from computation entirely.

5.4 How the pairs were built

Each pair (and each shared interpreter) was implemented by an *independent LLM agent* from a one-page registration brief, against the calculus as a written contract, under three standing rules: agents never modify the framework or another deliverable's code; interpreter changes are versioned events that re-validate every dependent pair (Proposition 4.7); and all growth is by additive widening — extend the covered fragment, re-run all prior evidence byte-for-byte, ratchet the coverage number. The platform's checks — the square oracle, the differentials, solver corroboration, branch agreement, twice-and-diff — were the only quality gate; no human reviewed agent code for semantic correctness. The defects those checks caught, and where they localized, are part of the evaluation (§6.5): the implementation process itself is an experiment in the paper's thesis that translation trust can be manufactured by architecture rather than by trusting the translator's author — whether that author is a compiler vendor or a language model.

6 Evaluation

We evaluate the snapshot along the axes the calculus makes measurable: per-pair conjoined coverage (§6.1), composed route coverage and branch agreement (§6.2), end-to-end case studies exercising Theorem 4.8 (§6.3), a certified-unreachability run inhabiting the `proved` tier (§6.4), the defects the architecture caught (§6.5), and performance and determinism (§6.6). Every number is regenerated by one script from the live registry and real runs at the snapshot commit (host: macOS/arm64, CPython 3.12, Z3 4.13, btormc 3.2, cadical; drat-trim pinned at source commit `2e3b2dc` and cake_lpr at `a4323b2`; the platform's own suite passes

Table 2. Capability snapshot: per-pair construct coverage (covered constructs / spec-derived inventory), declared fidelity grade, and the number of distinct typed `unsupported` gaps. Measured from the live registry at the snapshot commit.

Pair (source→target)	Grade	Coverage	Gaps
c-riscv	`reproducible`	—	—
riscv-btor2	`checked`	96/96	0
riscv-sail	`checked`	95/95	0
sail-btor2	`checked`	95/95	0
aarch64-btor2	`checked`	27/33	6
aarch64-sail	`checked`	27/33	6
wasm-btor2	`checked`	54/54	0
ebpf-btor2	`checked`	126/126	0
evm-btor2	`checked`	86/144	58
btor2-smtlib	`predicted`	56/56	0
crn-smtlib	`predicted`	10/10	0
python-smtlib	`predicted`	11/27	16
smiles-formula	`predicted`	14/17	3

1193 tests there). The evaluation is deliberately *limited but non-trivial*: small, fully-specified questions, every one checked end to end, rather than a large benchmark sweep.

6.1 Capability snapshot

Table 2 is the platform's honest self-portrait: per-construct coverage against spec-derived inventories (Definition 4.6), measured by running every probe through the pair's translator at harvest time — not transcribed from documentation. Depth is uneven by design and the unevenness is visible: the spine is deep (all 96 RV64IMC probe constructs; the eBPF in-scope set complete at 126; the BTOR2→SMT-LIB bridge total on its 56-item operator/sort/directive inventory), while EVM stands at 86 of the full 144-opcode inventory, Python at 11 of 27 subset constructs, and every gap is a *typed* `unsupported` the harness can enumerate (the "gaps" column). The C head carries no construct inventory: its translator is the pinned compiler, graded `reproducible` and re-established per run (Theorem 4.3).

6.2 Composed coverage and branch agreement

Table 3 (top) composes coverage along whole routes: a construct counts iff it survives *every* hop (Definition 4.6). The spine composes losslessly — 96/96 direct and 95/95 Sail-mediated RISC-V constructs survive to SMT-LIB, as do all in-scope eBPF, Wasm, and EVM constructs, and both AArch64 routes compose to the same 27/33 in-scope set — so per-pair depth is not being silently eaten at the hubs. The AArch64 rows also carry a ratchet anecdote the machinery makes precise: one revision earlier the Sail-mediated route composed to 0/33, because the Sail→BTOR2 translator lowered only the RV64IMC arm — and the harness localized every miss to the `sail-btor2` hop by name. Widening that one hop (an additive translator bump, RISC-V arm byte-identical

Table 3. Composed route coverage (a construct counts iff it survives every hop; "via Sail" routes are the independently derived branch), and branch agreement: the same question decided along both routes. Times are the slower route, end to end (translate every hop + decide with Z3).

Route (to SMT-LIB)	Composed coverage
RISC-V, direct	96/96
RISC-V, via Sail	95/95
AArch64, direct	27/33
AArch64, via Sail	27/33
Wasm	54/54
eBPF	126/126
EVM	86/144
CRN	10/10
Python	11/27
SMILES	14/17

Question (both routes)	Agree	Verdict	Time (s)
riscv const x1==42 (reach)	✓	reach	0.0
riscv const x1==99 (unreach)	✓	unreach	0.0
riscv loop sum==15 (reach)	✓	reach	0.0
riscv loop sum==99 (unreach)	✓	unreach	0.0
riscv store/load 0x123 (reach)	✓	reach	0.0
riscv store/load 0x999 (unreach)	✓	unreach	0.0
C a0==47 (reach)	✓	reach	0.1
C a0==99 (unreach)	✓	unreach	0.0
aarch64 movz/add x1==42 (reach)	✓	reach	0.0
aarch64 movz/add x1==999 (unreach)	✓	unreach	0.1
aarch64 SUBS/B.NE loop x0==1 (reach)	✓	reach	0.0
aarch64 SUBS/B.NE loop x0==5 (unreach)	✓	unreach	0.0
aarch64 flags + loop (SUBS/B.NE)	✓	trace $=_\pi$	—
aarch64 32-bit W forms	✓	trace $=_\pi$	—

per Proposition 4.7) closed the gap without re-earning any other evidence. A gap the system *reports and localizes* is the designed behavior; a gap papered over would be the defect.

The bottom of Table 3 is the fidelity payoff of Lemma 4.5: the same reachability questions decided along independently derived routes. Twelve solver-level questions — constant dataflow, summation and countdown loops under unrolling, store/load through memory, two questions about a *gcc-compiled C program* (can `a0` be 47 at halt? can it be 99?), and four AArch64 questions decided along the Arm-manual-derived and the Arm-Sail-model-derived route — were each decided along both routes of their branch; all twelve pairs of verdicts agree, including the universal (UNREACHABLE) halves where agreement carries real information. The C rows exercise the full re-establishment story: an opaque `reproducible` compiler heads both routes, and its output is validated downstream twice, independently. The AArch64 trace-level rows cross-check the same branch below the solver, directly comparing the two carried-back behaviors under π.

Table 4. End-to-end case studies. "Replay" is the source-level witness check of Theorem 4.8: the carried-back witness re-executed by the source interpreter.

Case	Verdicts	Replay	Time (s)
C spine (both routes + source replay)	REACHABLE (both agree)	✓	0.24
Python assert violable? (QF_LIA)	REACHABLE	✓	0.11
EVM 6*7==42 native vs bridged	REACHABLE (both agree)	✓	0.05
CRN A->B twice reaches B=2	REACHABLE	✓	3.56

6.3 End-to-end case studies

Table 4 instantiates Theorem 4.8 once per hub and front-end family. (1) The *C spine*: both routes answer REACHABLE for `a0`=47, and the source-level replay runs the compiled program in the shared RISC-V interpreter to the halt in `_start()` with `a0`=47 observed — the verdict survives even if every translator and the solver were wrong. (2) *Python*: "is the trailing assert violable?" for a function with a bounded loop and a branch; Z3 finds the witness x=4, the shared QF_LIA evaluator independently re-checks the model (`smt_model_ok`), and the input is replayed through the pinned CPython down the taken branches, firing the assert. (3) *EVM*: 6×7=42 on the operand stack, decided *natively* (btormc on the BTOR2 system, witness replayed by the strict BTOR2 interpreter) and *bridged* (through the SMT-LIB hub with Z3) — solve-step corroboration across two engine families, agreeing. (4) *CRN*: a two-firing reachability question in a reaction network, witness schedule replayed by the Petri-net interpreter. In all four, the final trust rests on the source interpreter alone, per the last row of the ledger in §4.4.

6.4 The certified tier

Table 5 inhabits the ledger's certified row (§4.4). The question is pair-derived and input-driven — an eBPF program whose `CALL` models the helper return as a free input x, squares it, and asks whether $x^2 = 3$; no square exists modulo 2^{64}, so the claim is universal over all inputs. Three engines corroborate UNSAT, bitwuzla bit-blasts the query to CNF, cadical emits a DRAT refutation, `drat-trim` elaborates it to LRAT — an *untrusted* step: a wrong elaboration can only fail the re-check, never fake a pass — and `cake_lpr`, the *formally verified* CakeML LRAT checker (soundness machine-proved down to the binary; here built natively from its compiler-generated ARMv8 assembly), re-validates the proof against the CNF from scratch. The verdict's TCB is exactly the ledger's certified row: the bit-blaster's BV→CNF step and a verified checker, with all three deciding engines and the elaborator demoted to discovery devices. The reachable sibling ($x^2 = 4$) correctly declines to certify.

Table 5. The certified (`proved`) row inhabited: an input-driven unreachability claim from a real pair, corroborated by three engines, certified by a bit-blasted DRAT proof, and re-validated by the independent checker. The controls are bogus checks that must fail — and do.

Question	eBPF: $helper()^2 = 3$ (no square mod 2^{64})
Engines agreeing UNSAT	bitwuzla, boolector, z3
Certificate	41746 B DRAT (bitwuzla→CNF, cadical→DRAT), elaborated to 18 B LRAT (drat-trim, untrusted)
Independent check	cake_lpr (*formally verified*): **verified**; tier `proved`
Resulting TCB	bitwuzla:bit-blast, cake_lpr:verified
Reachable sibling (x^2=4)	REACHABLE, no certificate
Negative controls	both rejected (bogus proofs vs. a satisfiable CNF)
Wall time	0.21 s

Wiring this exhibit produced one more architecture-caught defect, live: the checker *adapter* matched the substring `VERIFIED` in the checker's output — and `drat-trim` reports failure as `s NOT VERIFIED`, which contains it. Every check, sound or bogus, reported success. The bug could not surface on hosts without the checker (the call raised before parsing) nor on real certificates (which do verify); it fell to the *negative control* — a bogus refutation of a satisfiable CNF that must fail and did not. The fix parses the exact status line, and the control is now a permanent test. This is the paper's own instrument audited by the discipline it describes (cf. §6.5), and a caution we commend to anyone wiring a proof checker: *a checker adapter without a negative control is itself unchecked.* The verified checker's adapter was written under that rule from the start — wisely, since `cake_lpr` exits 0 even when checking fails; only the exact `s VERIFIED UNSAT` line signals success, and both checkers' negative controls are permanent tests. One honest subtlety the exercise surfaced: for this instance the refutation is propagation-dominated, so mutating single proof lines is not a valid negative control (the checker legitimately completes the derivation); the satisfiable-formula control is the sound one, since no valid refutation of a satisfiable formula exists.

6.5 Defects the architecture caught

The pairs were written by independent LLM agents with no human semantic review (§5.4); the architecture's checks were the only gate. Mining the full history (675 commits across all refs; ~21 runtime-code fix commits) yields 19 recorded defect incidents: 13 caught by the architecture's cross-checks rather than by ordinary unit tests, and 4 more — including the checker-adapter defect of §6.4 — by the audits and controls

Table 6. Defects caught by the architecture (selection; full list with commit-level evidence in the artifact). "Caught by" names the layer of §4.4 that fired.

Defect	Component	Caught by
bv64 sort hardcoded for `and`/`or`/`xor` over bv1 operands — malformed BTOR2 that real solvers tolerated	RISC-V → BTOR2 translator	strict in-process BTOR2 evaluator during the square's alignment walk
unsigned loads (`LBU`/`LHU`/`LWU`) missing zero-extension to bv64	RISC-V → BTOR2 translator	solver leg: Z3 sort rejection on the corpus
`init` value nid emitted after the state nid — rejected by every conformant tool, tolerated by the one wired solver; affected every stateful pair	shared BTOR2 emitter	solver corroboration: wiring the 2nd/3rd engines (btormc, pono) made the latent malformation manifest
`local.get` wrote a bv32 node into the bv64 stack array	Wasm → BTOR2 translator	solver leg: Z3 sort mismatch at BMC
carry-back silently zero-filled registers the solver witness omits (init-pinned states) — every pinned task's entry state misread	witness carry-back Λ	witness-replay anchor audit (mismatch chain)
default BMC bound mapped "no violation within k" to UNREACHABLE — a false negative at step 93	verdict semantics	oracle vs. corpus label
ELF header bytes decoded as instructions — interpreter tolerated it, the square did not	shared ELF loader	square oracle on the first real gcc binary
BTOR2 `constraint` lines silently dropped in the SMT-LIB unrolling — an under-constrained, soundness-leaking encoding	BTOR2 → SMT-LIB bridge	coverage probe: driving the bridge to its 56/56 spec inventory exposed the hole
`JUMP`/`JUMPI` underflow-halt row diverged on pc	EVM → BTOR2 translator	square step alignment during widening
`INT_MIN / -1`: C-undefined vs. RV64-defined divergence, plus a CBMC false positive on the same task	(real route divergence)	C-vs-ISA differential branch, with a UB-vs-fault classifier adjudicating

the architecture imposes on its own instruments. Table 6 lists ten representative architecture-caught defects, each localized — component, step, observable — as Corollary 3.8 promises.

Three second-order observations are worth as much as the table. First, *the checks caught their own instruments*, three times: the Sail ISA differential was once passing vacuously (comparing empty traces), the in-image suite caught a witness generator emitting empty witnesses, and the DRAT checker adapter accepted every outcome until a negative control ran (§6.4) — all found because the harnesses carry positive *and negative* controls, and all three are why Assumption 1 is stated as an assumption with empirical discharge rather than as a fact. Second, the history contains a genuine *blind spot* of the square, exactly of the class Lemma 4.5 predicts: the RISC-V interpreter *and* the RISC-V→BTOR2 translator both silently mis-decoded `MUL` (funct7 0x01) as `ADD` — both legs agreed, the square was blind, and the defect was found by manual audit. The platform's structural answer is diversity: the decoder is now anchored against 610 Sail-emitted words, and unknown funct7 hard-aborts on both sides. Third, in one incident three existing unit tests had been written *against the buggy behavior* (an evaluator masking every array store to 8 bits): tests written by a component's author codify the author's misreading; cross-checks against independently derived semantics do not. The architecture also *disconfirmed* one reported bug (a claimed slice-width defect shown to be a stale-cache artifact), and its differential campaigns — 300 seeded RV64IMC programs vs. the Sail simulator, a 10-seed Csmith campaign vs. native gcc, 463 reference cases — completed with zero divergences against harnesses whose positive controls confirm they can fail.

Table 7. Route cost for the RISC-V loop question (k=25 unrolling): whole-route translation, cold vs. warm (the content-addressed cache of Proposition 3.9), Z3 decide time, byte-determinism (twice-and-diff), and artifact size.

Route	cold (ms)	warm (ms)	decide (ms)	det.	artifact
RISC-V, direct	4	0.0	32	✓	185 kB
RISC-V, via Sail	5	0.1	32	✓	194 kB

6.6 Performance and determinism

Translation is not the cost center: whole-route translation of the loop question (k=25 unrolling, ~190 kB of SMT-LIB) is ~5 ms cold and a sub-millisecond content-addressed cache hit warm (Proposition 3.9 cashed in), against a Z3 decide of 30–70 ms; the twice-and-diff determinism check passes on both routes. The numbers say the honest overhead of the whole discipline — checking every hop, re-running for determinism, replaying witnesses — is small against the solver call it wraps.

7 Related work

Certified compilation and translation validation. CompCert [8] and CakeML [7] prove one translator correct once; translation validation [13, 15] and its modern descendants [9, 19] validate each run of one translator, and validators can themselves be verified [22]. In our vocabulary these are, respectively, the `proved` grade and the `checked` grade of a *single edge*. Our contribution is orthogonal: a calculus for a *graph* of edges of heterogeneous grade — how faithfulness, loss, and assurance compose along routes (Theorem 3.7, Proposition 4.2), how per-run validation re-establishes an opaque hop (Theorem 4.3), and how independent routes corroborate (Lemma 4.5). The commuting

square itself is the standard simulation/refinement diagram; the projection-compatibility hypothesis for pasting, the grade/class separation, and the trusted-base ledger are, to our knowledge, new as an explicit, composable discipline.

Semantics-first frameworks. The K framework derives interpreters, symbolic executors, and provers from one semantics [16], with KEVM [4] and KWasm as flagship instances; Sail does the same for ISAs [2]. These are our closest philosophical relatives, and our platform *consumes* their artifacts (the Sail RISC-V and Arm models anchor one of each ISA's two routes). The difference is what carries trust: a semantics-first framework is a monoculture — every derived tool inherits the one semantics and the one derivation pipeline — whereas our unit of trust is *agreement between independently derived translations* (Assumption 2), with the framework deliberately structured so routes share only endpoints. N-version programming [3] and differential testing [11, 24] exploit the same independence resource; we give the possibilistic account of what agreement does and does not establish (Lemma 4.5) and wire it into a compositional calculus rather than a test harness.

Certificates, witnesses, and certifying algorithms. Proof-carrying code [12], certifying algorithms [10], certified model-checking witnesses [14, 25], and checked solver proofs (DRAT [23], verified checkers [21], Alethe/Carcara [1]) all instantiate the producer/checker split our `proved` grade requires. Our end-to-end theorems lift the certifying-algorithm insight to translation graphs: an existential answer's certificate is its *source-level replay* after carry-back (Theorem 4.8) — making the target-to-source interpreter, usually an afterthought called a "lifter," the load-bearing component — while universal answers compose certificate checking with route fidelity (Theorem 4.9).

Moving programs to answerable representations. Verified lifting [5] and the broad practice of encoding programs into solver logics (BMC, symbolic execution) motivate the platform's existence; refinement stacks such as seL4's [6, 19] show multi-layer semantic preservation with uniform, proof-backed layers. Our setting differs in admitting layers that are *not* uniformly proved — pinned compilers, agent-written translators, third-party models — and making their heterogeneous trust first-class rather than a threat to the statement.

Fault tolerance, end to end. The paper's vocabulary is deliberately that of fault-tolerant distributed systems, because the concepts coincide: the square oracle is a failure detector that makes a silently wrong translator fail-stop [18]; a branch is dual modular redundancy whose comparator localizes rather than votes, with route diversity the classic defense against common-mode failure [3]; and the existential/universal asymmetry is the end-to-end argument [17] for translation graphs. What the transplant adds is the compositional bookkeeping — graded evidence with weakest-link meets, per-run re-establishment, and a ledger of what each check removes from the trusted base.

LLMs and formal tools. Recent systems couple LLM generation with solver-checked validation [20]. Our platform is built for that player — an LLM chooses questions and routes but every step it takes is deterministic, graded, and checked — and §5.4 reports the build half of the two-directional experiment of Section 1: the translators themselves were LLM-written, and correctness was carried by the architecture, not by author trust.

8 Limitations and conclusion

Limitations. This is a dated snapshot of a system designed to ratchet, and its numbers say so: coverage is deep on the spine (all of RV64IMC surviving composed to SMT-LIB; the in-scope eBPF set complete) and deliberately partial elsewhere (EVM at 86 of 144 opcodes; Wasm without `loop`/`br`; Python an integer subset with bounded unrolling). Universal answers are bounded claims wherever BMC-style unrolling caps apply. The `proved` tier's certified row is inhabited with a *formally verified* checker at the anchor (`cake_lpr`, §6.4); the residual TCB is the bit-blaster's BV→CNF step, and a certified claim is per-question and bounded, never a proof of a pair. Interpreter adequacy remains an assumption discharged empirically, and the diversity assumption behind branch corroboration is structural, not probabilistic. The compositional core of the calculus — including the n-ary route telescope — is mechanized in Lean 4 with a clean axiom audit (§4.7); the fieldwise loss computation and the retiming case remain paper-proved.

Conclusion. We proposed treating translations the way distributed systems treat processes: as components that fail, whose failures are contained by architecture — declared projections, decidable squares, graded evidence, independent routes, carried-back witnesses — rather than prevented by universal proof. The calculus makes the composition of partial trust exact; the platform shows the discipline is buildable, by agents, at useful coverage; and the asymmetry theorems say where proof effort actually buys assurance. The snapshot is the baseline; the ratchet is the roadmap.

References

[1] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.

[2] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 71:1–71:31.

[3] Algirdas Avižienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.

[4] Everett Hildenbrandt, Manasvi Saxena, Nishant Rodrigues, Xiaoran Zhu, Philip Daian, Dwight Guth, Brandon Moore, Daejun Park, Yi Zhang, Andrei Stefanescu, and Grigore Roşu. 2018. KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine. In *CSF*.
[5] Shoaib Kamil, Alvin Cheung, Shachar Itzhaky, and Armando Solar-Lezama. 2016. Verified Lifting of Stencil Computations. In *PLDI*.
[6] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *SOSP*.
[7] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.
[8] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.
[9] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.
[10] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.
[11] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
[12] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.
[13] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.
[14] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.
[15] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.
[16] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434.
[17] Jerome H. Saltzer, David P. Reed, and David D. Clark. 1984. End-to-End Arguments in System Design. *ACM Transactions on Computer Systems* 2, 4 (1984), 277–288.
[18] Richard D. Schlichting and Fred B. Schneider. 1983. Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems. *ACM Transactions on Computer Systems* 1, 3 (1983), 222–238.
[19] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *PLDI*.
[20] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807* (2024).
[21] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.
[22] Jean-Baptiste Tristan and Xavier Leroy. 2008. Formal Verification of Translation Validators: A Case Study on Instruction Scheduling Optimizations. In *POPL*.
[23] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.
[24] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*.
[25] Emily Yu, Armin Biere, and Keijo Heljanko. 2021. Progress in Certifying Hardware Model Checking Results. In *CAV*.

A Proofs

A.1 Pasting (Theorem 3.7)

We spell out the chain, including the domain side conditions elided in the body. Let $p \in \mathrm{dom}(P_2 \circ P_1)$, i.e. $p \in \mathrm{dom}(I_A) \cap \mathrm{dom}(T_1)$, $q := T_1(p) \in \mathrm{dom}(T_2)$, $r := T_2(q)$, $I_C(r)$ defined, $\Lambda_2(I_C(r)) \in \mathrm{dom}(\Lambda_1)$, and (from $p \in \mathrm{dom}(P_1)$, $q \in \mathrm{dom}(P_2)$) $I_B(q)$ defined with $I_B(q) \in \mathrm{dom}(\Lambda_1)$ and $I_C(r) \in \mathrm{dom}(\Lambda_2)$.

1. By faithfulness of P_1 at p and $\pi \subseteq \pi_1$, $\pi(I_A(p)) = \pi(\Lambda_1(I_B(q)))$.
2. By faithfulness of P_2 at q: $\pi_2(I_B(q)) = \pi_2(\Lambda_2(I_C(r)))$.
3. Both $I_B(q)$ and $\Lambda_2(I_C(r))$ lie in $\mathrm{dom}(\Lambda_1)$ (the first by $p \in \mathrm{dom}(P_1)$, the second by $p \in \mathrm{dom}(P_2 \circ P_1)$; the π_2-closure clause of Definition 3.6 makes the condition non-vacuous in general). By $(\pi_2 \Rightarrow \pi)$-support applied to step (2): $\pi(\Lambda_1(I_B(q))) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$.
4. Chaining (1) and (3): $\pi(I_A(p)) = \pi(\Lambda_1(\Lambda_2(I_C(r))))$, which is faithfulness of $P_2 \circ P_1$ at p w.r.t. π. □

Necessity of support. Without Definition 3.6 the theorem is false: let $F_B = \{g, h\}$, $\pi_2 = \{g\}$, and let Λ_1 copy field h into a π_1-kept source field. Take P_2 faithful (it preserves g) but with $\Lambda_2(I_C(r))$ differing from $I_B(q)$ on h — permitted, since $h \notin \pi_2$. The outer rectangle then disagrees on the copied field although both inner squares commute.

A.2 Composed keep-set, and step granularity

For fieldwise, step-aligned Λ_1 (each kept source field f computed per step as $f = \phi_f(\{\, b.g \mid g \in \mathit{deps}(f) \,\})$), $(\pi_2 \Rightarrow \pi)$-support holds iff $\mathit{deps}(f) \subseteq \pi_2$ for every $f \in \pi$; hence the largest admissible π is keep $= \{f \in \pi_1 \mid \mathit{deps}(f) \subseteq \pi_2\}$ and support for it is checkable syntactically from the dependency sets.

When Λ_1 retimes — one source step corresponds to a window of target steps, as in C statements over ISA instructions — model Λ_1 as a monotone window map w from source step indices to intervals of target step indices plus a fieldwise read within each window. Support then requires $\mathit{deps}(f) \subseteq \pi_2$ for reads *anywhere in the window*, and additionally that the window map itself be determined by π_2-observables (e.g. by a kept program counter), since window boundaries influence the projected output. Both conditions are again syntactic given w and the dependency sets.

A.3 Localization (Corollary 3.8)

Contrapositive of Theorem 3.7 for two hops. For n hops, induct on the route: if the outer rectangle over hops $1..n$ fails at p but hop 1's square passes at p, then by the two-hop case applied to (hop 1, rectangle over hops $2..n$) the rectangle over $2..n$ fails at $T_1(p)$; recurse. Termination yields a least failing hop i; within it, the square oracle's comparison yields the least failing step and a witnessing field. □

A.4 Weakest link (Proposition 4.2)

Soundness: if every hop is universal on its fragment, iterate Theorem 3.7 over all p in the composite fragment. If some hop is perrun, the same iteration is available exactly at programs whose per-hop images passed the oracle — the per-run set

of the route. If some hop is only replay, purity composes (Proposition 3.9) but no faithfulness statement is available for the route beyond it. Optimality: a route with a perrun hop entails no universal statement (take a defect outside the checked set); a route with a replay hop entails no faithfulness at all (take a pure but wrong translator). □

A.5 Ratchet (Proposition 4.7)

Let $I' \sqsupseteq I$ and let $p \in \mathrm{dom}(I)$. Every quantity in Definition 3.5 evaluated at p reads only I-values on $\mathrm{dom}(I)$-points, which I' preserves; hence the verdict at p is unchanged. Coverage monotonicity: cov's per-construct conjunction can only gain (new constructs enter dom) and never lose (old verdicts preserved). Composition of extensions is an extension. For the failure half: a non-extension update changes some $I(p)$, which can flip the verdict at p for any pair whose square consumed it — justifying the versioned-event rule. □

B Construct inventories and probe corpora

Per-language construct inventories (the yardsticks of Definition 4.6) and the probe corpora behind every coverage figure in Section 6 are enumerated in the artifact (`results/`); they are spec-derived: the RISC-V RV64IMC opcode inventory, the full 144-opcode EVM inventory, the 256-opcode eBPF inventory slice, the Wasm numeric/parametric/control inventory, the OpenSMILES construct list, the Python-subset AST allow-list, and the two hubs' operator inventories.